AI-Powered Medical Image Analysis – Pneumonia Detection from Chest X-Rays

By: Dhananjay Bhaskar

1. Introduction:

In today's world, hospitals generate a huge number of medical images every day, especially X-rays, CT scans and MRIs. Doctors need to carefully inspect each image to detect diseases, which is time-consuming and can be tiring. In this project, I built an AI system that looks at chest X-ray images and predicts whether the patient is normal or has pneumonia. The main idea is to use computer vision and deep learning to support doctors in detecting lung problems faster and with fewer mistakes.

Pneumonia is a serious lung infection that can cause fever, cough, chest pain and difficulty in breathing. It is especially dangerous for children, older people and patients with weak immune systems. Chest X-ray imaging is one of the most common ways to diagnose pneumonia, because it shows the condition of the lungs clearly. By training a Convolutional Neural Network (CNN) on real chest X-ray images, my model learns to recognize the visual patterns that separate healthy lungs from lungs affected by pneumonia.

This project does not replace doctors. Instead, it is meant to act like a smart assistant that can quickly screen X-ray images and highlight cases that look suspicious. This can help in early diagnosis, especially in places where there are fewer radiologists and many patients.

2. Problem Statement and Objectives

Problem Statement:

Doctors and radiologists often have to manually examine hundreds of X-ray images every day. Doing this for long hours can be slow and exhausting, and there is always a risk of human error. In many smaller hospitals, there may not even be enough experienced radiologists to read all images on time. This can delay diagnosis and treatment for diseases like pneumonia.

Objectives:

The main objectives of my project are:

1. To build an AI model that can analyze chest X-ray images and classify them as **Normal** or **Pneumonia**.
2. To train this model using a publicly available medical image dataset from Kaggle.
3. To evaluate the performance of the model using accuracy and a confusion matrix, so we can understand how well it is working.
4. To deploy the trained model as a simple web application using Flask, where a user can upload a chest X-ray image and get a prediction.

By completing these objectives, I was able to create a small but complete AI-powered medical image analysis system, starting from raw data and ending with a user-friendly website.

3. Dataset Description

For this project, I used the **Chest X-Ray Images (Pneumonia)** dataset from Kaggle. This dataset contains chest X-ray images from pediatric patients and has two classes: **NORMAL** and **PNEUMONIA**. The images are already organized into training, validation and test sets, which makes it suitable for building and evaluating a deep learning model.

In my project folder, the dataset is stored using the following structure:

- chest_xray/train/NORMAL
- chest_xray/train/PNEUMONIA
- chest_xray/val/NORMAL
- chest_xray/val/PNEUMONIA
- chest_xray/test/NORMAL
- chest_xray/test/PNEUMONIA

From the logs of my code, we can see the number of images used in each split:

- **Training set:** 5,216 images (NORMAL + PNEUMONIA)
- **Validation set:** 16 images (NORMAL + PNEUMONIA)
- **Test set:** 624 images (NORMAL + PNEUMONIA)

The training set is used to teach the CNN model, the validation set is used to monitor performance during training, and the test set is used at the end to check how well the model generalizes to unseen data.

4. Methodology

The complete pipeline of my project can be divided into several steps: data preprocessing, model design, model training, evaluation and deployment as a web application.

4.1 Data Preprocessing

The raw dataset contains X-ray images with different sizes. To feed them into a neural network, I first performed standard preprocessing:

1. **Resizing:** All images were resized to **256 × 256** pixels. This gives a fixed input size for the CNN.
2. **Grayscale conversion:** The X-ray images are effectively grayscale; in the pipeline I load them in grayscale so that the input shape is $256 \times 256 \times 1$.
3. **Normalization:** Pixel values were divided by 255 so that they lie in the range. This helps the model train faster and more stably.

To improve the robustness of the model, I also applied **data augmentation** using Keras ImageDataGenerator. The augmentation settings were:

- rotation_range = 10 degrees
- width_shift_range = 0.1
- height_shift_range = 0.1
- shear_range = 0.1
- zoom_range = 0.1
- horizontal_flip = True
- rescale = 1./255

The images were loaded and augmented using flow_from_directory(), which reads images directly from the train, val and test folders based on their class subfolders.

4.2 CNN Model Architecture

I designed a simple Convolutional Neural Network using TensorFlow/Keras. CNNs are very powerful for image classification because they automatically learn filters that detect edges, shapes and patterns in the images.

My CNN architecture is:

- **Conv2D layer:** 32 filters, 3×3 kernel, ReLU activation, input shape (256, 256, 1)
- **MaxPooling2D:** pool size 2×2

- **Conv2D layer:** 64 filters, 3×3 kernel, ReLU activation
- **MaxPooling2D:** pool size 2×2
- **Flatten layer:** converts the 2D feature maps into a 1D vector
- **Dense layer:** 128 neurons, ReLU activation
- **Dropout layer:** 0.5 (to reduce overfitting)
- **Output Dense layer:** 1 neuron, Sigmoid activation (for binary classification)

The model is compiled with:

- **Optimizer:** Adam
- **Loss function:** Binary Cross-Entropy
- **Metric:** Accuracy

This means the model outputs a probability between 0 and 1. Values closer to 1 indicate pneumonia, and values closer to 0 indicate normal.

4.3 Model Training

I trained the CNN model for **10 epochs** on the training set, while monitoring the accuracy and loss on the validation set after each epoch. During training, the model gradually improved its ability to distinguish between normal and pneumonia images.

From the training logs, we can see some example epoch results:

- Epoch 1: training accuracy ≈ 77.53%, validation accuracy ≈ 68.75%
- Epoch 5: training accuracy ≈ 90.97%, validation accuracy ≈ 75.00%
- Epoch 10: training accuracy ≈ 92.94%, validation accuracy ≈ 75.00%

The training accuracy kept increasing, while the validation accuracy fluctuated because the validation set is quite small (only 16 images). This shows that the model is learning but may also be slightly overfitting.

After training, I saved the model to a file called **medical_ai_model.h5**, which is later loaded by the evaluation script and the Flask app.

5. Evaluation and Results

After training the model, I evaluated it on the test set, which contains **624** images that the model has never seen during training. This gives a realistic measure of how well the model might perform on new data.

The evaluation results are:

- **Test Accuracy:** 79.49%

This means that on average, the model correctly classifies about 8 out of 10 X-ray images in the test set.

5.1 Confusion Matrix

To understand the types of mistakes the model makes, I calculated the confusion matrix using scikit-learn. The confusion matrix obtained was:

$$\begin{bmatrix} 112 & 122 \\ 6 & 384 \end{bmatrix}$$

Interpreting this:

- **True Normal predicted Normal:** 112
- **True Normal predicted Pneumonia (False Positive):** 122
- **True Pneumonia predicted Normal (False Negative):** 6
- **True Pneumonia predicted Pneumonia:** 384

In medical applications, false negatives (pneumonia predicted as normal) are more dangerous than false positives. It is better to wrongly flag a healthy person for further checks than to miss a sick person. In this model, there are only **6 false negatives**, which is relatively low compared to the number of pneumonia cases. However, there are more false positives (122), meaning the model sometimes predicts pneumonia even when the image is normal.

Overall, for a student-level project with a simple CNN architecture, an accuracy of around 79.5% and very few false negatives is a good starting point.

6. System Design and Web Application

The final part of the project is to deploy the trained model as a web-based tool so that users can interact with it easily. I used **Flask**, a lightweight Python web framework, to build a backend API and a simple frontend.

6.1 Backend (Flask API)

The Flask application exposes two main routes:

1. **/ (GET):** Renders the main web page (index.html) where the user can upload an X-ray image.

2. **/predict (POST):** Accepts an uploaded image file, preprocesses it and returns a prediction in JSON format.

The /predict route works as follows:

- Reads the uploaded image from the request.files object.
- Uses OpenCV to decode the bytes and convert the image to grayscale.
- Resizes the image to 256 × 256 and normalizes pixel values to.
- Reshapes the image to match the model input shape (1, 256, 256, 1).
- Passes the image to the loaded CNN model (medical_ai_model.h5).
- Gets the predicted probability, converts it into a label:
    - Probability > 0.5 → "Pneumonia Detected"
    - Probability ≤ 0.5 → "Normal"
- Returns a JSON response with the prediction and a confidence percentage.

6.2 Frontend (Web Interface)

On the frontend side, I created an HTML page using Tailwind CSS for styling. The interface contains:

- A title and short description of the tool.
- A drag-and-drop style file upload card for the chest X-ray image.
- An image preview area that shows the selected X-ray.
- A button labelled "Analyze X-ray".
- A result card that displays the predicted label (Normal or Pneumonia) along with the confidence percentage, using colored badges for better visual feedback.

The page uses a small JavaScript script that:

- Listens to the form submission.
- Sends the selected image to /predict using the Fetch API and FormData.
- Receives the JSON response and updates the result text and badge color accordingly.

This turns the project into a complete mini-application: a user can open the website, upload an X-ray, and immediately see the model's prediction.

7. Limitations

Even though the project works end-to-end, there are several limitations:

1. **Dataset size and variety:** The dataset is limited to one specific group of patients and only two classes (NORMAL and PNEUMONIA). Real hospitals may have more types of lung diseases and more diverse data.
2. **Model complexity:** The CNN used is relatively simple (two convolution layers). More advanced models like ResNet or DenseNet could achieve higher accuracy, but are heavier to train.
3. **Validation set size:** The validation set in this dataset contains only 16 images, so validation accuracy is not very stable and might not reflect the true generalization performance.
4. **Not clinically approved:** The system has not been tested or approved by medical experts, so it must not be used as a real diagnostic tool. It is only for learning and demonstration.

8. Future Scope

There are many ways this project can be improved in the future:

1. **Use transfer learning** by fine-tuning a pre-trained model (for example, ResNet-50) on the chest X-ray dataset, to improve accuracy and reduce training time.
2. **Add explainable AI techniques** like Grad-CAM to highlight which parts of the X-ray image the model is focusing on, so doctors can better trust the predictions.
3. **Extend to more classes**, such as COVID-19 or tuberculosis, by using larger multi-class chest X-ray datasets from Kaggle and other sources.
4. **Deploy to the cloud**, so that the web application is accessible from any device with internet, not just on my local machine.
5. **Add user accounts and history**, allowing doctors to log in, upload multiple images and see past predictions for the same patient.

9. Conclusion

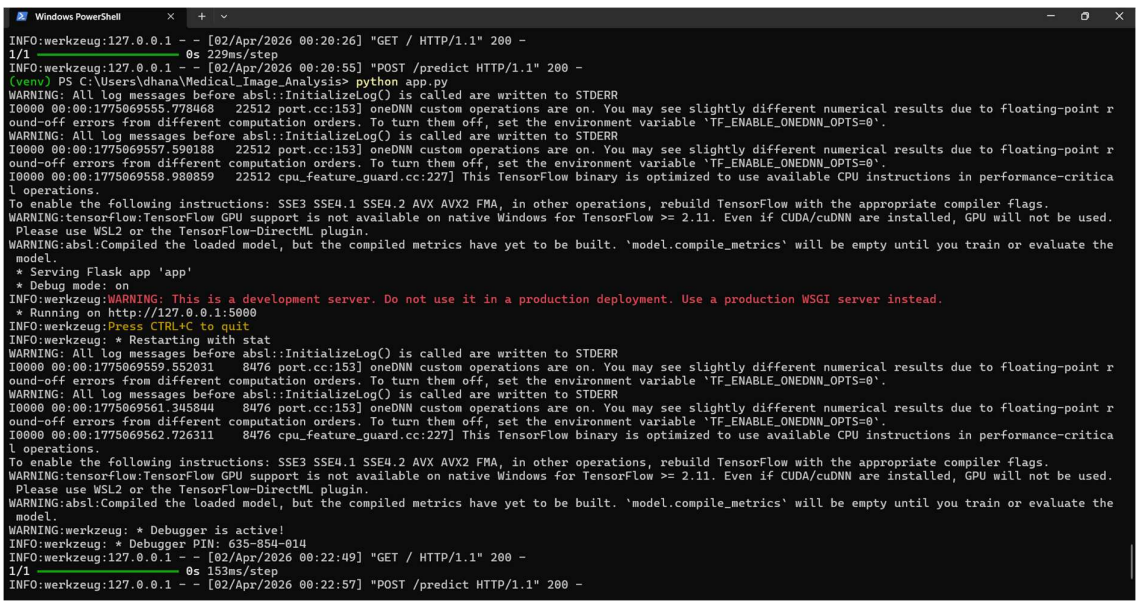
In this project, I successfully built an **AI-powered medical image analysis system** to detect pneumonia from chest X-ray images. Using the Chest X-Ray Images (Pneumonia) dataset from Kaggle, I preprocessed the images, trained a Convolutional Neural Network, evaluated

it with accuracy and a confusion matrix, and finally deployed it as a Flask-based web application.

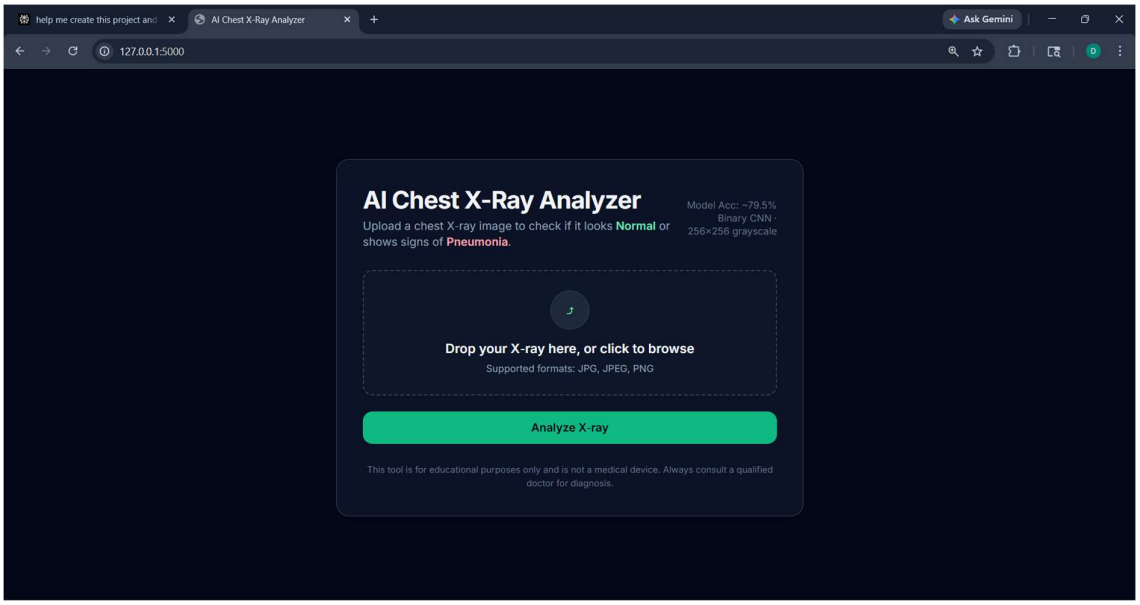
The model achieved around **79.49% test accuracy** and produced very few false negatives, which is important in medical screening applications. Although it is not perfect and cannot be used as a real medical device, this project clearly demonstrates how deep learning and web technologies can be combined to build practical tools for healthcare.

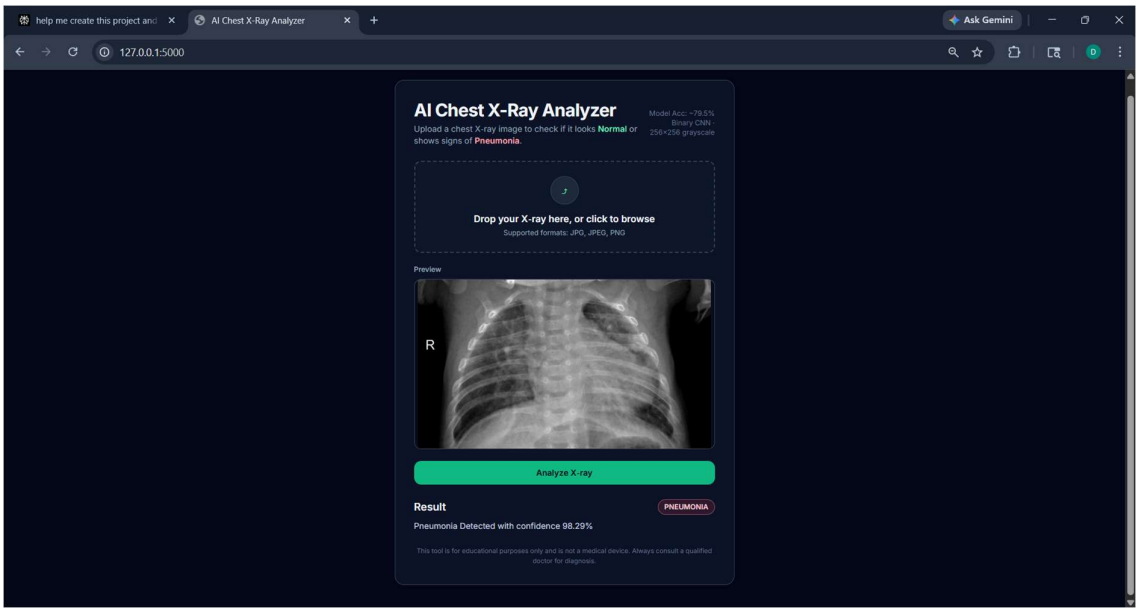
Working on this project taught me the complete workflow of an AI project: collecting data, preprocessing, model design, training, evaluation, and deployment. It also showed me the challenges of working with medical data and the importance of explaining and validating AI models before using them in real life. In the future, I would like to improve this system using transfer learning and explainable AI, and experiment with more diseases and datasets.

10. Sample outputs:



```
Windows PowerShell
INFO:werkzeug:127.0.0.1 - - [02/Apr/2026 00:20:26] "GET / HTTP/1.1" 200 -
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 229ms/step
INFO:werkzeug:127.0.0.1 - - [02/Apr/2026 00:20:55] "POST /predict HTTP/1.1" 200 -
(venv) PS C:\Users\dhana\Medical_Image_Analysis> python app.py
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1775069555.778468   22512 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point r
ound-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1775069557.590188   22512 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point r
ound-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1775069558.980859   22512 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critica
l operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used.
 Please use WSL2 or the TensorFlow-DirectML plugin.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty until you train or evaluate the
 model.
 * Serving Flask app 'app'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with stat
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1775069559.552031    8476 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point r
ound-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1775069561.345844    8476 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point r
ound-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1775069562.726311    8476 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critica
l operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used.
 Please use WSL2 or the TensorFlow-DirectML plugin.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty until you train or evaluate the
 model.
WARNING:werkzeug: * Debugger is active!
INFO:werkzeug: * Debugger PIN: 635-854-014
INFO:werkzeug:127.0.0.1 - - [02/Apr/2026 00:22:49] "GET / HTTP/1.1" 200 -
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 153ms/step
INFO:werkzeug:127.0.0.1 - - [02/Apr/2026 00:22:57] "POST /predict HTTP/1.1" 200 -
```


help me create this project and
AI Chest X-Ray Analyzer
Ask Gemini
127.0.0.1:5000
AI Chest X-Ray Analyzer
Upload a chest X-ray image to check if it looks Normal or shows signs of Pneumonia.
Model Acc: ~79.5%
Binary CNN ·
256×256 grayscale
Drop your X-ray here, or click to browse
Supported formats: JPG, JPEG, PNG
Analyze X-ray
This tool is for educational purposes only and is not a medical device. Always consult a qualified doctor for diagnosis.


help me create this project and
AI Chest X-Ray Analyzer
Ask Gemini
127.0.0.1:5000
AI Chest X-Ray Analyzer
Upload a chest X-ray image to check if it looks Normal or shows signs of Pneumonia.
Model Acc: ~79.5%
Binary CNN ·
256×256 grayscale
Drop your X-ray here, or click to browse
Supported formats: JPG, JPEG, PNG
Preview
R
Analyze X-ray
Result
PNEUMONIA
Pneumonia Detected with confidence 98.29%
This tool is for educational purposes only and is not a medical device. Always consult a qualified doctor for diagnosis.